**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ**

**Факультет физико-математических и естественных наук**

**Кафедра прикладной информатики и теории вероятностей**

# ОТЧЕТ
# ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

*дисциплина: Архитектура компьютера*

Студент: Лебеденко Елена Викторвна

Группа: НКАбд-04-25

**МОСКВА**

2025 г.

# Содержание

1. Цель работы……..……4

2. Выполнение лабораторной работы…….…….….6-14

2.1 Реализация подпрограмм в NASM….…….….……6

2.2 Отладка программам с помощью GDB…...……..……..…….7

2.2.1 Добавление точек останова……..……..……..……...10

2.2.2 Работа с данными программы в GDB...…….……..……...10

2.2.3 Обработка аргументов командной строки в GDB..……..…….……13

3. Задания для самостоятельной работы……...…….15-19

4. Вывод……..……20

# Список иллюстраций

1.1 Создание каталога и файла для выполнения л/р……....6

1.2 Вывод программы из листинга 9.1…….….6

1.3 Изменённая программа из листинга 9.1.…….……....6

1.4 Вывод программы из листинга 9.1……...…….……...7

1.5 Отладка программы из листинга 9.2…..……...……...7

1.6 Установка брейкпоинта на метку _start...……...……8

1.7 Дисассимилированный код программы..…….……8

1.8 Дисассимилированный код программы с Intel’овским синтаксисом..…….…9

1.9 Режим псевдографики……...……...…9

1.10 Проверка с помощью команды info breakpoints...…….……..10

1.11 Установка точки останова по адресу инструкции…….…….10

1.12 Команда info registers……....…….……...10

1.13 Значения переменных msg1 и msg2……....…….……11

1.14 Замена символа в msg1……...…….…..…11

1.15 Замена символа в msg2……...…….…..…11

1.16 Проверка с помощью команды info breakpoints...…….……..12

1.17 Команда p/*……....…….……..12

1.18 Команды continue и quit……....…….…..…12

1.19 Создание файла lab09-3.asm……....…….…...…13

1.20 Загрузка исполняемого файла в отладчик…....…….……..…13

1.21 Точка останова……....……..…13

1.22 Регистр esp……....…….……14

1.23 Разные позиции стека……....…….…..…14

2.1 Задание 1(1)…….……..……...…….15

2.2 Задание 1(2)…….……..……...……15

2.3 Задание 2(1)…….……..……...……17

2.4 Задание 2(2)…….……..……...……18

2.5 Задание 2(3)…….……..……...…18
2.6 Задание 2(4)…….……..……..19

## 1. Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

# 2. Выполнение лабораторной работы

## 2.1. Реализация подпрограмм в NASM

1) Создаю каталог для программам лабораторной работы № 9 перехожу в него и создаю файл lab09-1.asm

```
evlebedenko@evlebedenko:~$ mkdir ~/work/arch-pc/lab09
evlebedenko@evlebedenko:~$ cd ~/work/arch-pc/lab09
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ touch lab09-1.asm
```

1.1 Создание каталога и файла для выполнения л/р

Ввожу в файл lab9-1.asm текст программы из листинга 9.1, создаю исполняемый файл и запускаю его.

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nano lab09-1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 2
2x+7=11
```

1.2 Вывод программы из листинга 9.1

Изменяю текст программы, добавив подпрограмму _subcalcul в подпрограмму _calcul, для вычисления выражения $f(g(x))$, где $x$ вводится с клавиатуры, $f(x)=2x+7$, $g(x)=3x-1$. Т.е. $x$ передается в подпрограмму _calcul из нее в подпрограмму _subcalcul, где вычисляется выражение $g(x)$, результат возвращается в _calcul и вычисляется выражение $f(g(x))$.

```
_calcul:
push eax
call _subcalcul
mov ebx,2
mul ebx
add eax,7

mov [res],eax
pop eax
ret ; выход из подпрограммы

_subcalcul:
mov ebx,3
mul ebx
sub eax,1

ret
```

1.3 Изменённая программа из листинга 9.1

Создаю исполняемый файл и проверьте его работу.

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nano lab09-1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 2
2(3x-1)+7=17
```

1.4 Вывод программы из листинга 9.1

## 2.2 Отладка программам с помощью GDB

Создаю файл lab09-2.asm в каталоге ~/work/arch-pc/lab09, ввожу в него текст программы из листинга 9.2 (Программа печати сообщения Hello world!) и создаю исполняемый файл. Для работы с GDB добавляю отладочную информацию, для этого трансляцию программ необходимо проводить с ключом '-g'. Загружаю в отладчик gdb и проверяю работу программы, запустив ее в оболочке GDB с помощью команды run.

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /home/evlebedenko/work/arch-pc/lab09/lab09-2
Hello, world!
[Inferior 1 (process 19879) exited normally]
(gdb)
```

1.5 Отладка программы из листинга 9.2

Для более подробного анализа программы устанавливаю брейкпоинт на метку _start, с которой начинается выполнение любой ассемблерной программы, и запускаю её

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/evlebedenko/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9       mov eax, 4
(gdb)
```

1.6 Установка брейкпоинта на метку _start

Смотрю дисассимилированный код программы с помощью команды disassemble начиная с метки _start

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:     mov    $0x4,%eax
   0x08049005 <+5>:     mov    $0x1,%ebx
   0x0804900a <+10>:    mov    $0x804a000,%ecx
   0x0804900f <+15>:    mov    $0x8,%edx
   0x08049014 <+20>:    int    $0x80
   0x08049016 <+22>:    mov    $0x4,%eax
   0x0804901b <+27>:    mov    $0x1,%ebx
   0x08049020 <+32>:    mov    $0x804a008,%ecx
   0x08049025 <+37>:    mov    $0x7,%edx
   0x0804902a <+42>:    int    $0x80
   0x0804902c <+44>:    mov    $0x1,%eax
   0x08049031 <+49>:    mov    $0x0,%ebx
   0x08049036 <+54>:    int    $0x80
End of assembler dump.
```

1.7 Дисассимилированный код программы

Переключаюсь на отображение команд с Intel’овским синтаксисом, введя команду set disassembly-flavor intel

Различия отображения синтаксиса машинных команд в режимах ATT и Intel:

- Порядок операндов (в ATT источник-назначение, в Intel назначение-источник)

- Регистры с префиксом % в ATT

- Константы с префиксом $ в ATT

```
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:     mov    eax,0x4
   0x08049005 <+5>:     mov    ebx,0x1
   0x0804900a <+10>:    mov    ecx,0x804a000
   0x0804900f <+15>:    mov    edx,0x8
   0x08049014 <+20>:    int    0x80
   0x08049016 <+22>:    mov    eax,0x4
   0x0804901b <+27>:    mov    ebx,0x1
   0x08049020 <+32>:    mov    ecx,0x804a008
   0x08049025 <+37>:    mov    edx,0x7
   0x0804902a <+42>:    int    0x80
   0x0804902c <+44>:    mov    eax,0x1
   0x08049031 <+49>:    mov    ebx,0x0
   0x08049036 <+54>:    int    0x80
End of assembler dump.
```

1.8 Дисассимилированный код программы с Intel’овским синтаксисом

Включаю режим псевдографики для более удобного анализа программы

```
Register group: general
eax            0x0                 0
ecx            0x0                 0
edx            0x0                 0
ebx            0x0                 0
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0

B+>0x8049000 <_start>      mov    eax,0x4
   0x8049005 <_start+5>    mov    ebx,0x1
   0x804900a <_start+10>   mov    ecx,0x804a000
   0x804900f <_start+15>   mov    edx,0x8
   0x8049014 <_start+20>   int    0x80
   0x8049016 <_start+22>   mov    eax,0x4

native process 20299 (asm) In: _start                    L9     PC: 6
(gdb) layout regs
(gdb)
```

1.9 Режим псевдографики

### 2.2.1 Добавление точек останова

На предыдущих шагах была установлена точка останова по имени метки (_start). Проверяю это с помощью команды info breakpoints (кратко i b)

```
(gdb) info breakpoints
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x08049000 lab09-2.asm:9
        breakpoint already hit 1 time
```

1.10 Проверка с помощью команды info breakpoints

Определяю адрес предпоследней инструкции (mov ebx,0x0), установливаю точку останова, а также смотрю информацию о всех установленных точках останова.

```
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 22.
(gdb) i b
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x08049000 lab09-2.asm:9
        breakpoint already hit 1 time
2       breakpoint     keep y   0x08049031 lab09-2.asm:22
```

1.11 Установка точки останова по адресу инструкции

### 2.2.2 Работа с данными программы в GDB

Смотрю содержимое регистров с помощью команды info registers

```
ecx            0x804a000          134520832
edx            0x8                8
ebx            0x1                1
esp            0xffffd000         0xffffd000
ebp            0x0                0x0
esi            0x0                0

B+ 0x8049000 <_start>      mov    $0x4,%eax
   0x8049005 <_start+5>    mov    $0x1,%ebx
   0x804900a <_start+10>   mov    $0x804a000,%ecx
   0x804900f <_start+15>   mov    $0x8,%edx
  >0x8049014 <_start+20>   int    $0x80
   0x8049016 <_start+22>   mov    $0x4,%eax
   0x804901b <_start+27>   mov    $0x1,%ebx

native process 22250 (asm) In: _start                    L13   PC: 0x804
eax            0x4                4
ecx            0x804a000          134520832
edx            0x8                8
ebx            0x1                1
esp            0xffffd000         0xffffd000
ebp            0x0                0x0
esi            0x0                0
edi            0x0                0
```

1.12 Команда info registers

Смотрю значение переменной msg1 по имени, а также смотрю значение переменной msg2 по адресу.

```
   0x804900f <_start+15>   mov    edx,0x8
   0x8049014 <_start+20>   int    0x80
   0x8049016 <_start+22>   mov    eax,0x4
   0x804901b <_start+27>   mov    ebx,0x1
   0x8049020 <_start+32>   mov    ecx,0x804a008
   0x8049025 <_start+37>   mov    edx,0x7
   0x804902a <_start+42>   int    0x80
   0x804902c <_start+44>   mov    eax,0x1
native process 22868 (asm) In:  _start
ds             0x2b                43
es             0x2b                43
fs             0x0                 0
gs             0x0                 0
(gdb) x/1sb &msg1
0x804a000 <msg1>:       "Hello, "
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:       "world!\n\034"
```

1.13 Значения переменных msg1 и msg2

Изменяю первый символ переменной msg1

```
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:       "hello, "
```

1.14 Замена символа в msg1

Заменяю любой символ во второй переменной msg2

```
(gdb) set {char}0x804a008='w'
(gdb) x/1sb &0x804a008
Attempt to take address of value not located in memory.
(gdb) x/1sb &msg2
0x804a008 <msg2>:       "world!\n\034"
```

1.15 Замена символа в msg2

Вывожу в различных форматах (в шестнадцатеричном формате, в двоичном формате и в символьном виде) значение регистра edx.

```
Register group: general
eax            0x0                 0
ecx            0x0                 0
edx            0x0                 0
ebx            0x0                 0
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0

B+>0x8049000 <_start>      mov    eax,0x4
   0x8049005 <_start+5>    mov    ebx,0x1
   0x804900a <_start+10>   mov    ecx,0x804a000
   0x804900f <_start+15>   mov    edx,0x8
   0x8049014 <_start+20>   int    0x80
   0x8049016 <_start+22>   mov    eax,0x4

native process 22868 (asm) In: _start
(gdb) p/s $edx
$1 = 0
(gdb) p/L $edx
No symbol "L" in current context.
(gdb) p/x $edx
$2 = 0x0
```

1.16 Команда p/*

С помощью команды set изменяю значение регистра ebx

```
(gdb) set $ebx='2'
(gdb) p/s $ebx
$3 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$4 = 2
```

1.17 Изменение значения ebx

В первом случае выводится ASCII-символ, а во втором — числовое значение

Завершаю выполнение программы с помощью команды continue и выхожуиз GDB с помощью команды quit.

```
(gdb) c
Continuing.
hello, world!

Breakpoint 2, _start () at lab09-2.asm:22
(gdb) q
```

1.18 Команды continue и quit

### 2.2.3 Обработка аргументов командной строки в GDB

Копирую файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем lab09-3.asm и создаю исполняемый файл.

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/a
rch-pc/lab09/lab09-3.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
```

1.19 Создание файла lab09-3.asm

Для загрузки в gdb программы с аргументами необходимо использовать ключ — args. Загружаю исполняемый файл в отладчик, указав аргументы.

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb)
```

1.20 Загрузка исполняемого файла в отладчик

Установливаю точку останова перед первой инструкцией в программе и запускаю ее.

```
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 8.
(gdb) run
Starting program: /home/evlebedenko/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:8
8       pop ecx         ; Извлекаем из стека в `ecx` количество
```

1.21 Точка останова

Адрес вершины стека храниться в регистре esp и по этому адресу располагается число равное количеству аргументов командной строки (включая имя программы

```
(gdb) x/x $esp
0xffffcfc0:     0x00000005
```

1.22 Регистр esp

Смотрю остальные позиции стека – по адесу [esp+4] располагается адрес в памяти где находиться имя программы, по адесу [esp+8] храниться адрес первого аргумента, по адресу [esp+12] – второго и т.д.

```
(gdb) x/s *(void**)($esp + 4)
0xffffd191:     "/home/evlebedenko/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)($esp + 8)
0xffffd1be:     "аргумент1"
(gdb) x/s *(void**)($esp + 12)
0xffffd1d0:     "аргумент"
(gdb) x/s *(void**)($esp + 16)
0xffffd1e1:     "2"
(gdb) x/s *(void**)($esp + 20)
0xffffd1e3:     "аргумент 3"
(gdb) x/s *(void**)($esp + 24)
0x0:    <error: Cannot access memory at address 0x0>
```

1.23 Разные позиции стека

Размер шага равен 4, так как на 32-битной архитектуре размер указателя = 4 байта. Т.е, чтобы перейти на следующую строку, нужно прибавить 4 байта.

## 3. Задание для самостоятельной работы

1)Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму.

Напишите программу, которая находит сумму значений функции $f(x)$ для $x = x1, x2, ..., xn$, т.е. программа должна выводить значение $f(x1) + f(x2) + ... + f(xn)$. Значения $xi$ передаются как аргументы. Вид функции $f(x)$ выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах $x = x1, x2, ..., xn$.

Вариант 16: $30x - 11$

Копирую программу задания в новый файл

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/task.asm ~/work/
arch-pc/lab09/task1.asm
```

2.1 Задание 1 (1)

Переделываю программу

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nano task1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf task1.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o task1 task1.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ./task1
Функция: f(x)=30x-11
Функция: f(x)=30x-110
```

2.2 Задание 1(2)

## Листинг:

```
%include 'in_out.asm'

SECTION .data
msgf db "Функция: f(x)=30x-11",0
msg db "Результат: ",0
```

```
SECTION .text
global _start

_start:
mov eax, msgf
call sprintLF

pop ecx
pop edx
sub ecx,1
mov esi,0

next:
cmp ecx,0h
jz _end
pop eax
call atoi

call _fx

add esi, eax
loop next
_end:
mov eax, msgf
call sprint
mov eax, esi
call iprintLF
call quit
```

_fx:

mov ebx,30

mul ebx

sub eax,11

2) В листинге 9.3 приведена программа вычисления выражения (3 + 2) * 4 + 5. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Попробую запустить программу

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf task2.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o task2 task2.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ./task2
Результат: 10
```

2.3 Задание 2(1)

Действительно, ответ не верный

С помощью si построчно проверю изменения значений регистров.

Проблема возникает при выполнение инструкции *mul ecx*. Перед этим мы берём число 3 (*mov ebx,3*), число 2 (*mov eax,2*) и складываем их в регистр ebx (*add ebx,eax*). Инструкция *mul ecx* умножает число 4, записанное в ecx, на значение в регистре eax (т. е. 2) и записыват новое значение (2*4=8) в eax.

```
evlebedenko@evlebedenko: ~/work/arch-pc/lab09
Register group: general
eax            0x2                 2
ecx            0x4                 4
edx            0x0                 0
ebx            0x5                 5
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0
esi            0x0                 0

B+ 0x80490e8 <_start>      mov    ebx,0x3
   0x80490ed <_start+5>    mov    eax,0x2
   0x80490f2 <_start+10>   add    ebx,eax
   0x80490f4 <_start+12>   mov    ecx,0x4
  >0x80490f9 <_start+17>   mul    ecx
   0x80490fb <_start+19>   add    ebx,0x5
   0x80490fe <_start+22>   mov    edi,ebx

native process 26328 (asm) In: _start                    L12   PC: 0x80490f9
eax            0x2                 2
ecx            0x4                 4
edx            0x0                 0
ebx            0x5                 5
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0
esi            0x0                 0
edi            0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--
```

2.4 Задание 2(2)

```
evlebedenko@evlebedenko: ~/work/arch-pc/lab09
Register group: general
eax            0x8                 8
ecx            0x4                 4
edx            0x0                 0
ebx            0x5                 5
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0
esi            0x0                 0

   0x80490f2 <_start+10>   add    ebx,eax
   0x80490f4 <_start+12>   mov    ecx,0x4
   0x80490f9 <_start+17>   mul    ecx
  >0x80490fb <_start+19>   add    ebx,0x5
   0x80490fe <_start+22>   mov    edi,ebx
   0x8049100 <_start+24>   mov    eax,0x804a000
   0x8049105 <_start+29>   call   0x804900f <sprint>

native process 26328 (asm) In: _start                    L13   PC: 0x80490fb
ebx            0x5                 5
esp            0xffffd000          0xffffd000
ebp            0x0                 0x0
esi            0x0                 0
edi            0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) si
(gdb)
```

2.5 Задание 2(3)

Чтобы исправить программу, я инструкцию *mov ebx,eax* на *mov eax,ebx* и заменяю далее все ebx на eax. В итоге получаю правильную программу

```
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nano task2.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ nasm -f elf task2.asm
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ld -m elf_i386 -o task2 task2.o
evlebedenko@evlebedenko:~/work/arch-pc/lab09$ ./task2
Результат: 25
```

2.6 Задание 2(4)

Листинг:

```
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add eax,ebx
mov ecx,4
mul ecx
add eax,5
mov edi,eax
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

# 4. Вывод

При выполнении лабораторной работы я приобрела навыки написания программ с использованием подпрограмм и познакомилась с методами отладки при помощи GDB и его основными возможностями.